Email Integration

This document describes how the system integrates with email for caregiver sign-in/out notifications.

Email Notifications

Purpose

When a member is signed in or out, an email notification can be sent to their caregiver. This keeps caregivers informed of their child's arrival and departure in real time.

How It Works

The email notification system is in `kiosk_signin/email_notifier.py`. It uses Python's built-in `smtplib` and `email` modules to send emails via Gmail's SMTP server.

Configuration

Email settings are stored in `kiosk_signin/email_config.json`:

```
{
   "enabled": true,
   "sender_email": "your-email@gmail.com",
   "app_password": "your-app-password"
}
```

The `app_password` is a Gmail App Password, not the account's main password. App Passwords are generated in Google Account settings under Security > 2-Step Verification > App Passwords.

Trigger Flow

1. After a successful sign-in/out, `email_notifier.send_notification()` is called. This happens both on the kiosk (`signature_screen.py`) and on the web dashboard's Sign In / Out page (`/signin` in `dashboard/app.py`), including bulk sign in/out, once per person signed.
2. The function spawns a daemon thread to send the email asynchronously, so it does not block the UI.
3. Inside the thread, it: - Loads the email config from the JSON file. - Checks if notifications are enabled. - Looks up the member's caregiver and their email address. - Checks the caregiver's `notify_on` preference ("NONE", "IN", "OUT", or "BOTH"). - Constructs and sends the email if all conditions are met.

Threading

The email is sent in a background daemon thread:

```
def send_notification(member, direction, timestamp):
   threading.Thread(
```

```
        target=_send, args=(member, direction, timestamp), daemon=True
    ).start()
```

Daemon threads are automatically terminated when the main program exits, so they do not prevent the application from closing. The `daemon=True` flag is important here because SMTP connections can hang if the server is unreachable, and we do not want that to block shutdown.

Error Handling

All exceptions during email sending are silently caught. This is intentional -- email notifications are a best-effort feature and should never crash the sign-in process or display errors to the caregiver at the kiosk.

Email Content

The email is sent as a plain text MIME message:

```
Subject: Kiosk Sign-In Notification -- Ava Thompson

Hi,

Ava Thompson has been signed IN at 03:15 PM on Monday 21 June 2026.

-- Kiosk Sign-In System
```

Notification Preferences

Each caregiver can configure when they receive notifications via the `notify_on` field:

- **BOTH** -- Notified on both sign-in and sign-out (default).
- **IN** -- Notified only on sign-in.
- **OUT** -- Notified only on sign-out.
- **NONE** -- No notifications.

This is set on the member's profile screen (kiosk) or the person edit form (dashboard).

Programming Concepts Applied

Asynchronous I/O with Threading

Sending email uses Python's `threading.Thread` with `daemon=True` for non-blocking I/O. This is a common pattern in GUI applications where the main thread must remain responsive while network operations run in the background.

Configuration Files

Credentials are stored in a JSON file loaded at runtime. This separates configuration from code and allows credentials to be changed without modifying the source.

MIME Message Construction

The email system uses Python's `email.mime` module to construct properly formatted email messages. `MIMEMultipart` creates the message container, and `MIMEText` attaches the plain text body. This follows the MIME standard for email formatting.